

SC102 L08: 软体物理模拟——详细学习笔记

涵盖：2D 链条效果 | 质点弹簧系统 | 胡克定律 | 欧拉方法 | 绳索模拟 | 布料模拟

1 软体物理概述

[p.4]

软体物理 (Soft Body Physics) 解决的是对可变形物体进行实时物理模拟的问题。与刚体物理不同：

- 刚体：物体有明确不变的形状，位置 + 旋转就能完全描述
- 软体：没有固定的外形界限，核心挑战是**形变 (Deformation)**

常见的软体模拟应用：衣物布料、头发、绳索、水袋、果冻等。

2 2D 链条：使用 Unity 关节的”伪软体”

[p.5]

先使用 Unity 自带物理引擎制作软体效果，理解核心思想。

制作藤蔓/锁链步骤：

1. 将藤蔓分成约 5 段，每段是一个 Independent GameObject
2. 每段添加 Rigidbody2D 和碰撞盒
3. 每段添加 HingeJoint2D，连接到其上方的一段
4. 最顶端的段固定在一个锚点

原理：利用 HingeJoint2D”围绕一点旋转”的特性——每个关节提供一个旋转自由度，整条链在重力下自然下垂弯曲，形成软体效果。

限制：虽然能弯曲，但不能像弹簧一样拉伸收缩——本质是”无弹性的质点弹簧系统”。

3 质点弹簧系统 (Mass Spring System)

[p.6-7]

为了写出更真实可控的软体模拟，需要从物理学入手。

构成软体系统的三大要素：

1. 分子间引力：保证软体不会像流体一样散开
2. 分子间斥力：保证软体体积不变，不会凭空变大变小
3. 重力与弹力：使软体会下落并盖在其他物体上

质点弹簧系统的核心组成：

- 质点 (Mass Point)：具有质量的点，位置 + 速度
- 弹簧 (Spring)：连接两个质点的无质量线段，有自然长度和弹性系数
- 缓冲器 (Damper)：控制”回弹速度”，防止无限振荡

4 胡克定律 (Hooke's Law)

[p.8]

胡克定律定义了弹簧的运动规律：

$$F = k \cdot x$$

其中：

- F ：拉伸/压缩弹簧所需的力（力的方向始终指向恢复自然长度）
- k ：弹性系数 (Spring Constant)， k 越大弹簧越”硬”
- x ：弹簧当前长度与自然长度的差值 ($x = L_{\text{current}} - L_{\text{rest}}$)

关键：弹簧被拉得越长，回弹力就越大；弹性系数越大，弹簧越”结实”（更难拉伸）。

5 单个质点弹簧的模拟

[p.9-10]

模拟需要 5 个状态量：

1. **Rest Length**：弹簧自然状态下的长度
2. **Current Length**：弹簧当前（被拉伸后）的长度

3. **Spring Constant (k):** 弹性系数

4. **Force (F):** 弹簧受到的力, $F = k \cdot x$

5. **Acceleration (a):** $a = F/m$ (牛顿第二定律, 这里设 $m = 1$ 简化)

```
Vector3 force = Vector3.zero;
float currentLength = Vector3.Distance(springPos, massPos);
if (currentLength != restLength)
    force += (currentLength - restLength) * -springConstant
            * (massPos - springPos).normalized;
Vector3 acceleration = force;           // m=1简化
velocity += acceleration * Time.deltaTime;
velocity *= (1 - damping);              // 缓冲系数
mass.position += velocity * Time.deltaTime;
```

代码详解:

- `(currentLength - restLength)`: x , 当前拉伸量
- `-springConstant`: 负号使力始终指向恢复方向
- `.normalized`: 力的方向 (从弹簧固定点指向质点)
- `acceleration = force`: 设质量 $m = 1$, 则 $a = F$
- `velocity *= (1-damping)`: 缓冲系数 (0 到 1), 模拟能量耗散

6 欧拉方法 (Euler Method)

[p.11]

`mass.position += velocity * Time.deltaTime` 是一种近似算法——欧拉方法(显式欧拉法)。

欧拉方法公式:

$$y_{n+1} = y_n + y'_n \cdot h$$

其中 y 是位置, y' 是速度 (位置的导数), h 是时间步长 (`Time.deltaTime`)。

误差来源: 欧拉方法假设速度在时间步内保持不变。当时间步长较大或速度变化剧烈时, 误差显著。在软体模拟中这可能导致“数值爆炸”——弹簧过度回弹、质点飞出去。

改进方法:

- **半隐式欧拉 (Semi-Implicit Euler):** 先更新速度再更新位置 (较稳定)
- **Verlet 积分:** 不显式存储速度, 用当前位置和上一位置推算
- **Runge-Kutta 方法 (RK4):** 在一个步长内采样 4 次, 更高精度

7 绳索软体模拟 (1D)

[p.12]

将单个质点弹簧扩展到一维：生成多个质点排成一线，每个质点受左右两个质点的弹簧力影响。

```
// 每个质点计算左右弹簧力
var leftForce = springConstant * (leftDist - spacing)
                * (leftSpherePos - myPos).normalized;
force += leftForce;
var rightForce = springConstant * (rightDist - spacing)
                * (rightSpherePos - myPos).normalized;
force += rightForce;
```

可视化：使用 `LineRenderer` 组件连接所有质点位置，绘制绳索曲线。

8 布料模拟 (2D)

[p.13-15]

在绳索基础上增加一个维度形成 2D 网格，每个质点受周围 4 个质点的力影响。

网格可视化：使用 `Mesh` 类构建平面模型，顶点位置设为质点位置：

```
int vert = 0;
for (int i = 0; i < width; i++)
    for (int j = 0; j < height; j++)
        vertices[vert++] = spheres[i, j].transform.position;
mesh.vertices = vertices;
mesh.RecalculateNormals();
```

加入重力与碰撞：

- 为每个质点添加 `Collider` 和 `Rigidbody`，让它们自然下落
- 用 `rigidbody.MovePosition()` 替代 `Transform.Translate()`——后者不会触发物理引擎的碰撞响应

当前方法的限制：

1. 质点数量有限，影响物理模拟精度
2. 欧拉方法在高复杂度系统中容易不稳定
3. Unity 内置物理引擎不完全可控，可能导致意外行为

9 课后习题详解

[p.16]

9.1 题 1：固定布料某些点的位置

思路：对于需要固定的质点，每帧将其位置强制设为目标位置，并在计算弹簧力时跳过更新其速度。

```
void UpdateCloth() {
    for (int i = 0; i < width; i++)
        for (int j = 0; j < height; j++) {
            if (IsFixedPoint(i, j)) {
                // 固定点：不参与物理计算，位置保持
                spheres[i,j].position = fixedPositions[i,j];
                velocities[i,j] = Vector3.zero;
                continue;
            }
            // 正常物理更新...
        }
}
```

9.2 题 2：HingeJoint2D 藤蔓平台

步骤：

1. 用 5-6 段物体 + HingeJoint2D（自顶向下连接）制作藤蔓链
2. 最顶端固定在场景锚点
3. 最底端连接平台（Rigidbody2D+Collider2D）
4. 给所有段添加适当质量（上面的轻、下面的越来越重模拟平台重量）
5. 玩家跳上平台时，自身重量会造成藤蔓晃动（HingeJoint 动态响应）

9.3 题 3：电线杆之间的电线

实现：使用质点弹簧绳索模型，两端质点固定（position 锁定在电线杆位置），中间质点受重力和弹簧力自然下垂形成悬链线。角色碰撞检测到电线时将对应质点向下推，角色离开后弹簧力自动恢复。